

COURSE 3 - DATA ENGINEERING 2026**C3-060 - Distributed Processing with Spark**

Beginner-first teacher edition: understand Spark from the physical mental model, then build, break, inspect and prove it.

Artifact	C3-060
Status	2026 - LEARNER EDITION
Phase	C3-05 Distributed Processing with Spark 16 lessons 46-60 focused hours
Prerequisite	C3-04 Gate PASS; C2B SQL/Python/modeling foundations assumed, distributed execution taught here.
Current lane	Apache Spark / PySpark 4.2.0 Python 3.10+ Java 17+ learner local[2]
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: mental picture -> tiny worked example -> guided real Spark workflow -> expected result -> why it worked -> break/fix -> independent changed task -> validation -> physical-plan/production reasoning -> evidence. A green Spark job is not mastery if grain, quality, plan or reproducibility is wrong.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-05-L01 Why distributed processing: partitions, executors and coordination	6
C3-05-L02 Spark architecture, driver and execution model	9
C3-05-L03 PySpark DataFrames and explicit schemas	12
C3-05-L04 Selecting, filtering and expression design	15
C3-05-L05 Joins and distributed join consequences	18
C3-05-L06 Aggregations, grouping and window functions	21
C3-05-L07 Nulls, types and data-cleaning patterns	24
C3-05-L08 Reading and writing CSV/JSON/Parquet/Delta	27
C3-05-L09 Spark SQL and DataFrame interoperability	30
C3-05-L10 Partitions, repartition, coalesce and shuffles	33
C3-05-L11 Query plans, explain and execution-stage reasoning	36
C3-05-L12 Caching and persistence decisions	39
C3-05-L13 Data skew and uneven partition diagnosis	42
C3-05-L14 Adaptive execution and tuning concepts	45
C3-05-L15 Testing Spark transformations and output invariants	48
C3-05-L16 Packaging and running repeatable Spark jobs	51
Mastery checklist + revision quick reference	54

Module map + protected learning route

Distributed Processing with Spark

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

How this phase works

C3-05 is not "learn PySpark syntax." It teaches you to keep business correctness while reasoning about distributed execution. Every lesson uses the same three-page teacher loop so the interface never changes as concepts become harder.

Local labs intentionally use local[2] and tiny shuffle settings so an 8 GB learner laptop can practice semantics. Those settings are teaching scaffolds, not production recommendations.

Current technical baseline

- Apache Spark / PySpark 4.2.0 is the current stable lane for this rebuild.
- PySpark supports Python 3.10+ and requires Java 17+ with JAVA_HOME available.
- AQE is enabled by default in Spark SQL; local exercises may set `spark.sql.shuffle.partitions=4` only to keep tiny demonstrations understandable.
- PySpark includes `assertDataFrameEqual` and `assertSchemaEqual` for testing.
- Delta is not assumed to be installed with PySpark; C3-04 owns Delta fundamentals and C3-05 treats Delta I/O as an optional runtime integration.

Study loop

READ mental model -> WATCH optional visual when useful -> FOLLOW once -> predict expected result -> RUN/inspect -> BREAK one assumption -> FIX from evidence -> DO changed task -> VALIDATE grain/quality/plan -> EXPLAIN -> SAVE evidence.

Exact phase route

L01-L16 -> matching P01-P16 -> protected R01-R16 only after genuine attempt -> CourierSpark Mini-Lab -> MeterSpark Gate A -> targeted repair if needed -> fresh B/C only by explicit full-retest assignment.

C3-05-L01 - Why distributed processing: partitions, executors and coordination

Focused effort: 2.5 h | Matching practice: P01 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Explain when distributed processing is justified and reason from partitions, tasks, stages and jobs rather than from "Spark is faster."

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a large warehouse order split into boxes. Each box is a partition. Workers process boxes in parallel, but a coordinator still decides which boxes belong to each step. If every box must be reshuffled before the next step, coordination and network movement become real costs.

5. New words before we use them

Partition = a chunk of distributed data processed by one task at a time. Task = one unit of work for one partition in a stage. Stage = tasks that can run without a shuffle boundary. Job = work triggered by an action. Shuffle = redistribution of data across partitions.

6. Useful source / visual

[Apache Spark 4.2.0 documentation](#)

[RECOMMENDED VISUAL: Spark architecture video](#)

Useful segment: Driver/workers, partitions, jobs/stages/tasks and shuffle overview.

C3-05-L01 - Follow with me once

7. Tiny worked example / prediction

Sketch a 1 TB scan split into input partitions, then mark a groupBy shuffle boundary and the downstream stage.

8. Follow with me once - real project action

Write architecture/L01_execution_map.md for the supplied order pipeline. Label driver coordination, input partitions, tasks, shuffle boundary and output partitions.

9. Expected result

A correct execution map that distinguishes job, stages, tasks and partitions and identifies the shuffle boundary.

10. Why it worked

Production choice: use distributed processing when data size, throughput, shared platform or parallel workload justifies its overhead. The evidence is not the word "big"; it is observed resource/runtime/operational need.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you call rows "tasks" or assume every transformation launches a job, inspect the action boundary and redraw.

C3-05-L01 - Now prove it yourself

13. Common beginner mistakes

- Assuming Spark is faster for every dataset.
- Calling rows "tasks" or partitions "executors."
- Tuning before stating input/output grain.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Change the example to 80 MB and argue whether local pandas could be more appropriate.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: use distributed processing when data size, throughput, shared platform or parallel workload justifies its overhead. The evidence is not the word "big"; it is observed resource/runtime/operational need.

18. Teach it back

Explain why distributed processing: partitions, executors and coordination in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P01 evidence first. Then open C3-062 R01. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

execution map + changed-size decision. You can explain partition/task/stage/job from a fresh pipeline and state one distributed-overhead cost.

C3-05-L02 - Spark architecture, driver and execution model

Focused effort: 3.0 h | Matching practice: P02 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Separate driver responsibility from distributed executor/task work, explain lazy evaluation and avoid driver-heavy designs.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a film director and several camera crews. The driver plans and coordinates; executors perform tasks on their local partitions. Transformations build the shot list lazily; an action says "record now." Bringing every raw frame back to the director is like an unbounded collect().

5. New words before we use them

Driver = process that coordinates the Spark application and builds plans. Executor = worker-side process that runs tasks and stores cached/shuffle data. Lazy evaluation = transformations are recorded before execution. Action = operation that triggers execution.

6. Useful source / visual

[Spark 4.2.0 RDD programming guide - cluster concepts](#)

[RECOMMENDED VISUAL: Spark architecture video](#)

Useful segment: Focus on driver/executor/lazy/action mental model; video is optional.

C3-05-L02 - Follow with me once

7. Tiny worked example / prediction

Trace read -> filter -> join -> groupBy -> write. Identify which calls build lineage and which action materializes work.

8. Follow with me once - real project action

Complete docs/L02_driver_safety.md and identify unsafe patterns in src/anti_patterns.py.example.

9. Expected result

A driver-safety review that correctly separates lazy transformations, actions and bounded/unbounded driver collection.

10. Why it worked

Production choice: keep the driver responsible for coordination, not bulk data collection. Anything returned to the driver must have a documented upper bound.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you think collect() is always wrong, refine: it is acceptable only when you have proven the result is bounded and intentionally driver-sized.

C3-05-L02 - Now prove it yourself

13. Common beginner mistakes

- Using collect()/toPandas() without a proven bound.
- Confusing lazy transformation code with executed work.
- Treating local[2] as a production cluster.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Review a second pipeline that collects distinct customer IDs and decide safe/unsafe from an explicit upper bound.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: keep the driver responsible for coordination, not bulk data collection. Anything returned to the driver must have a documented upper bound.

18. Teach it back

Explain spark architecture, driver and execution model in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P02 evidence first. Then open C3-062 R02. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

driver-safety review + changed transfer. You can identify driver risks and lazy/action boundaries without guessing.

C3-05-L03 - PySpark DataFrames and explicit schemas

Focused effort: 3.0 h | Matching practice: P03 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Build a laptop-safe SparkSession, read typed data with an explicit StructType and save schema/row/partition evidence.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a typed shipping manifest. A DataFrame is not just rows: each column has a type and nullability expectation. An explicit schema is the contract that says what may enter before Spark begins distributed work.

5. New words before we use them

DataFrame = distributed typed table-like dataset. StructType/StructField = explicit schema objects. Nullable = whether null is allowed. SparkSession = entry point used to create/read DataFrames and submit work.

6. Useful source / visual

[PySpark 4.2.0 installation](#)

[RECOMMENDED VISUAL: Data Engineering Zoomcamp 2026 Spark module](#)

Useful segment: Use the setup/DataFrame lesson linked from the module; exact repository module may evolve.

C3-05-L03 - Follow with me once

7. Tiny worked example / prediction

Use local[2], 1g driver memory, four shuffle partitions and AQE enabled; read orders.csv with ORDER_SCHEMA.

8. Follow with me once - real project action

Complete src/schemas.py TODOs and scripts/01_read_orders_TODO.py. Save printSchema, row count and getNumPartitions evidence when runtime is available.

9. Expected result

A Spark 4.2.0 local session that can read the provided source with an explicit schema and report schema, row count and partition count.

10. Why it worked

Production choice: schema is an interface contract. Keep raw text when needed to distinguish parsing errors from legitimate nulls and version the schema when it changes.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If a type mismatch becomes null, separate legitimate null from parse failure instead of silently accepting it.

C3-05-L03 - Now prove it yourself

13. Common beginner mistakes

- Relying on schema inference for a contractual source.
- Treating cast-to-null as valid missing data.
- Skipping Java/Python/Spark version verification.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Read a second shipment fixture with an explicit schema and compare nullable rules.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: schema is an interface contract. Keep raw text when needed to distinguish parsing errors from legitimate nulls and version the schema when it changes.

18. Teach it back

Explain pyspark dataframes and explicit schemas in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P03 evidence first. Then open C3-062 R03. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

schema code + runtime evidence + transfer. You can create a fresh typed DataFrame and defend each required/nullability choice.

C3-05-L04 - Selecting, filtering and expression design

Focused effort: 2.5 h | Matching practice: P04 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Express transformations with Spark Column/built-in functions so logic remains visible to Catalyst and stays distributed.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a conveyor belt where labels are changed while boxes stay on the belt. Built-in Column expressions let Spark see and optimize the full belt. Pulling each box off into a Python loop breaks that visibility and adds crossing overhead.

5. New words before we use them

Column expression = Spark expression object evaluated by the engine. Built-in function = optimizer-visible Spark SQL function. UDF = user-defined function; use only when built-ins cannot express the logic. Catalyst = Spark SQL optimizer.

6. Useful source / visual

[PySpark DataFrame API](#)

[OPTIONAL VISUAL: Data Engineering Zoomcamp Spark module](#)

Useful segment: Use the DataFrames/SQL demonstration when a visual walkthrough helps.

C3-05-L04 - Follow with me once

7. Tiny worked example / prediction

Derive `net_amount` and `high_value`, filter valid paid rows, and keep rejected rows separate.

8. Follow with me once - real project action

Complete `clean_orders()` in `src/transforms_TODO.py` without `collect()`, `toPandas()` or a Python UDF.

9. Expected result

Clean and rejected DataFrames produced with built-in Spark expressions while preserving one clean row per valid `order_id`.

10. Why it worked

Production choice: prefer optimizer-visible expressions. A UDF must have a clear reason, tests, and a measured cost if it is on a critical path.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you reach for a row loop, rewrite the rule as Column expressions; if impossible, document why a UDF is justified.

C3-05-L04 - Now prove it yourself

13. Common beginner mistakes

- Using Python loops/UDFs when built-ins exist.
- Adding withColumn repeatedly without checking plan/readability.
- Dropping invalid rows instead of routing them.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Add a changed discount rule and prove row grain remains one row per order_id.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: prefer optimizer-visible expressions. A UDF must have a clear reason, tests, and a measured cost if it is on a critical path.

18. Teach it back

Explain selecting, filtering and expression design in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P04 evidence first. Then open C3-062 R04. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

transformation + row-grain validation. You can implement fresh column logic and explain why it remains distributed.

C3-05-L05 - Joins and distributed join consequences

Focused effort: 3.5 h | Matching practice: P05 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Choose a join from business grain, prove key cardinality first, then reason about broadcast versus shuffle strategies.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture two warehouses that must match parcels by product_id. If the dimension has duplicate product IDs, one parcel can multiply into several output rows. Only after grain is proven should you decide whether to broadcast the small side or shuffle both sides.

5. New words before we use them

Join grain = what one row represents before/after the join. Broadcast join = send a sufficiently small side to executors. Shuffle join = repartition data by join keys. Fan-out = unintended row multiplication.

6. Useful source / visual

[PySpark DataFrame.join](#)

[RECOMMENDED VISUAL: Data Engineering Zoomcamp - internals of GroupBy and joins](#)

Useful segment: Watch the join-internals segment linked from the module; preserve grain checks from the book.

C3-05-L05 - Follow with me once

7. Tiny worked example / prediction

Prove products.product_id unique; join orders to products; compare expected input/output row counts; inspect join plan when runtime is available.

8. Follow with me once - real project action

Complete join_products() and evidence/L05_join_contract.md. Reject duplicate dimension keys before joining.

9. Expected result

A join that keeps expected fact grain, blocks duplicate dimension keys and can be explained as broadcast-or-shuffle from evidence.

10. Why it worked

Production choice: enforce key cardinality before join, then choose strategy from observed relation sizes and plan/runtime evidence. Correctness is not a tuning option.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If totals inflate, do not add DISTINCT. Inspect dimension uniqueness and join grain.

C3-05-L05 - Now prove it yourself

13. Common beginner mistakes

- Fixing join multiplication with DISTINCT.
- Broadcasting by guess rather than size/evidence.
- Skipping dimension-key uniqueness checks.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Use a deliberately duplicated product dimension and show how the validation blocks the join.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: enforce key cardinality before join, then choose strategy from observed relation sizes and plan/runtime evidence. Correctness is not a tuning option.

18. Teach it back

Explain joins and distributed join consequences in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P05 evidence first. Then open C3-062 R05. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

join code + uniqueness control + plan note. You can prove a fresh join is grain-safe and explain its likely physical consequence.

C3-05-L06 - Aggregations, grouping and window functions

Focused effort: 3.5 h | Matching practice: P06 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Build grouped metrics with explicit output grain and window metrics that preserve row grain.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture two views of the same queue. groupBy compresses many rows into one summary row per group. A window attaches a calculation to each original row while looking across related rows. If you do not state output grain, the two can be confused.

5. New words before we use them

Aggregation = reduction to grouped summary rows. groupBy = define grouping keys. Window = calculation over related rows while retaining row detail. PartitionBy/orderBy = window grouping and ordering rules.

6. Useful source / visual

[PySpark Window API](#)

[RECOMMENDED VISUAL: Data Engineering Zoomcamp Spark module](#)

Useful segment: Use the DataFrames/SQL material for aggregation/window reinforcement.

C3-05-L06 - Follow with me once

7. Tiny worked example / prediction

Compute region revenue (region grain) and customer_order_number using Window (order grain).

8. Follow with me once - real project action

Complete regional_metrics() and add_customer_sequence(). Record before/after row counts and grains.

9. Expected result

A region-grain aggregate plus an order-grain window result with deterministic ordering and explicit before/after grain.

10. Why it worked

Production choice: write grain beside each transformation. For windows with ties, define deterministic ordering if the result depends on rank/sequence.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If a window unexpectedly changes rows, inspect joins/filters around it; window itself should not collapse rows like groupBy.

C3-05-L06 - Now prove it yourself

13. Common beginner mistakes

- Forgetting that groupBy changes grain.
- Using an unstable window order with ties.
- Validating only total row count after aggregation.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Create a product-category rank per region while preserving one row per order.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: write grain beside each transformation. For windows with ties, define deterministic ordering if the result depends on rank/sequence.

18. Teach it back

Explain aggregations, grouping and window functions in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P06 evidence first. Then open C3-062 R06. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

aggregation + window + grain checks. You can state grain before and after any group/window transformation.

C3-05-L07 - Nulls, types and data-cleaning patterns

Focused effort: 3.0 h | Matching practice: P07 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Separate legitimate nulls from parse failures, quarantine invalid records and avoid silent data loss.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture an intake desk with three trays: valid, legitimately missing, and malformed. A nullable field belongs in the "allowed missing" tray; bad text that failed a numeric cast belongs in quarantine, not silently in the same null bucket.

5. New words before we use them

Parse failure = raw value cannot be converted to required type. Business null = intentionally missing value allowed by contract. Quarantine = rejected records kept with reason. Reconciliation = clean + rejected equals source.

6. Useful source / visual

[Spark SQL null semantics](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-05-L07 - Follow with me once

7. Tiny worked example / prediction

Use raw string columns to detect malformed qty/unit_price before typed curated output. Keep rejected reason codes.

8. Follow with me once - real project action

Complete validate_orders() and produce clean/rejected logic without dropping bad rows invisibly.

9. Expected result

Source rows split into clean and rejected sets with reason codes and exact source = clean + rejected reconciliation.

10. Why it worked

Production choice: preserve bad records and reason codes. Data quality is part of the pipeline contract, not an afterthought after Spark says SUCCESS.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If casts yield null, compare raw text and required-field policy before fillna/coalesce.

C3-05-L07 - Now prove it yourself

13. Common beginner mistakes

- fillna(0) on parse failures.
- Dropping malformed rows silently.
- Mixing missing and invalid values into one reason.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Add a new malformed timestamp row and ensure it is rejected with a reason rather than converted to business null.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: preserve bad records and reason codes. Data quality is part of the pipeline contract, not an afterthought after Spark says SUCCESS.

18. Teach it back

Explain nulls, types and data-cleaning patterns in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P07 evidence first. Then open C3-062 R07. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

clean/rejected rules + reconciliation. You can distinguish missing, invalid and nullable data on changed input.

C3-05-L08 - Reading and writing CSV/JSON/Parquet/Delta

Focused effort: 3.0 h | Matching practice: P08 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Read supported formats deliberately and write healthy Parquet output while explaining Delta runtime boundaries.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture different freight containers. CSV/JSON are easy interchange formats, Parquet stores analytical columns efficiently, and Delta adds table-level transaction semantics over files. Spark uses similar read/write APIs, but the operational contract is not identical.

5. New words before we use them

Parquet = columnar analytical file format. Codec = compression method. Save mode = overwrite/append/error/ignore behavior. Delta = transactional table format used with an additional runtime/package, not automatically included just because PySpark is installed.

6. Useful source / visual

[Spark Parquet data source](#)

[OPTIONAL VISUAL: Data Engineering Zoomcamp Spark module](#)

Useful segment: Use the Parquet/repartition exercise as visual reinforcement.

C3-05-L08 - Follow with me once

7. Tiny worked example / prediction

Read CSV + JSON fixtures, write curated Parquet, then specify the Delta writer call without pretending Delta exists in a plain Spark runtime.

8. Follow with me once - real project action

Complete scripts/02_orders_job_TODO.py output path logic and docs/L08_io_contract.md.

9. Expected result

Correctly typed reads and a healthy Parquet write whose row/schema controls match the source contract; Delta use is explicitly conditional on the runtime/package.

10. Why it worked

Production choice: treat file format, partitioning, save mode and transactional-table support as separate decisions. Validate file counts/sizes and table semantics after write.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you create one file per record or coalesce(1) by habit, reconsider output scale and downstream parallelism.

C3-05-L08 - Now prove it yourself

13. Common beginner mistakes

- Writing Parquet and assuming partition/file health is good.
- Confusing Spark's Parquet support with automatic Delta support.
- Using append/overwrite without an explicit publish contract.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Write a second domain to Parquet with a different partition strategy and justify it.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: treat file format, partitioning, save mode and transactional-table support as separate decisions. Validate file counts/sizes and table semantics after write.

18. Teach it back

Explain reading and writing csv/json/parquet/delta in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P08 evidence first. Then open C3-062 R08. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

I/O code + output strategy + Delta boundary. You can build a fresh I/O path and state format/runtime assumptions honestly.

C3-05-L09 - Spark SQL and DataFrame interoperability

Focused effort: 2.5 h | Matching practice: P09 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Translate between DataFrame and Spark SQL forms while preserving semantics, grain and validation.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture two translators describing the same plan: the DataFrame API and Spark SQL. Both can compile into Spark's optimizer. The goal is not to prefer one religion; it is to prove they produce the same grain, filters, joins and values.

5. New words before we use them

Temp view = session-scoped SQL name backed by a DataFrame. Semantic parity = two implementations return equivalent data at the same grain. SQL optimizer = planning layer shared by SQL and DataFrame expressions.

6. Useful source / visual

[Spark SQL programming guide](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-05-L09 - Follow with me once

7. Tiny worked example / prediction

Register clean orders as a temp view; reproduce region metrics in spark.sql; compare values to DataFrame output.

8. Follow with me once - real project action

Complete scripts/04_sql_equivalence_TODO.py and save an equality/reconciliation check.

9. Expected result

SQL and DataFrame versions of one metric return equivalent columns/values at the same grain.

10. Why it worked

Production choice: teams may mix SQL and DataFrame APIs. Require semantic parity, code review and shared tests rather than style arguments.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If SQL and DataFrame differ, compare filters/null semantics/grouping columns before blaming the engine.

C3-05-L09 - Now prove it yourself

13. Common beginner mistakes

- Assuming SQL and DataFrames differ in correctness by default.
- Comparing outputs with different null/filter semantics.
- Creating temp views and calling them governed tables.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Translate a changed high-value metric both ways and compare.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: teams may mix SQL and DataFrame APIs. Require semantic parity, code review and shared tests rather than style arguments.

18. Teach it back

Explain spark sql and dataframe interoperability in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P09 evidence first. Then open C3-062 R09. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

two implementations + parity check. You can prove semantic equivalence on fresh logic.

C3-05-L10 - Partitions, repartition, coalesce and shuffles

Focused effort: 3.5 h | Matching practice: P10 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Use partition controls intentionally and identify where data redistribution creates Exchange/shuffle work.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture moving people between buses. repartition can redistribute everyone into new buses, which costs movement; coalesce usually reduces buses with less movement. The number of buses affects parallel work and output files, but "one bus" is not a universal optimization.

5. New words before we use them

repartition = full redistribution that can increase/decrease partitions. coalesce = usually reduce partitions with less movement. Exchange = plan node indicating data movement. Output file count = often related to final partitions, but not a guaranteed one-to-one contract in every sink/runtime.

6. Useful source / visual

[Spark SQL performance tuning](#)

[RECOMMENDED VISUAL: Spark UI performance walkthrough](#)

Useful segment: Use partition/task/stage views; visual is optional support.

C3-05-L10 - Follow with me once

7. Tiny worked example / prediction

Start with four partitions, repartition by region, aggregate, then inspect formatted plan and output partition/file counts.

8. Follow with me once - real project action

Complete scripts/03_explain_join_TODO.py and evidence/L10_partition_notes.md. Never use `coalesce(1)` as the default answer.

9. Expected result

A deliberate repartition/coalesce experiment with partition counts, output-file evidence and at least one visible Exchange where redistribution occurs.

10. Why it worked

Production choice: partition counts are workload-specific. Use plan, task, file-size and skew evidence; do not freeze tiny local settings as production defaults.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If fewer partitions makes runtime worse, consider lost parallelism, skew and stage boundaries.

C3-05-L10 - Now prove it yourself

13. Common beginner mistakes

- Using coalesce(1) as a universal fix.
- Changing partition count without measuring output/plan effects.
- Confusing input partitions with post-shuffle partitions.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Choose a partition plan for a changed dataset with one hot region and four cold regions.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: partition counts are workload-specific. Use plan, task, file-size and skew evidence; do not freeze tiny local settings as production defaults.

18. Teach it back

Explain partitions, repartition, coalesce and shuffles in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P10 evidence first. Then open C3-062 R10. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

plan evidence + partition decision. You can connect partition APIs to physical movement and output layout.

C3-05-L11 - Query plans, explain and execution-stage reasoning

Focused effort: 3.5 h | Matching practice: P11 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Read formatted plans for scans, filters/projects, exchanges, joins, aggregates, windows and adaptive markers.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture an X-ray of the job. `explain("formatted")` shows scans, filters, exchanges, joins and aggregates. It does not prove business correctness, but it lets you connect code to physical work before guessing at tuning knobs.

5. New words before we use them

Logical plan = relational operations before physical choices. Physical plan = concrete operators Spark plans to execute.

Exchange = shuffle boundary. Join strategy = broadcast/hash/sort-merge choice. Adaptive plan = runtime-adjusted plan under AQE.

6. Useful source / visual

[PySpark DataFrame.explain](#)

[RECOMMENDED VISUAL: Query plan to Spark SQL UI performance](#)

Useful segment: Focus on scans, exchanges, joins and statistics in the SQL tab.

C3-05-L11 - Follow with me once

7. Tiny worked example / prediction

Call `explain("formatted")` on the join+aggregation pipeline and annotate at least one Exchange and the join operator.

8. Follow with me once - real project action

Complete `evidence/L11_plan_annotation.md` from real output when Spark is available.

9. Expected result

A formatted plan annotation that connects Scan/Filter/Exchange/Join/Aggregate/Adaptive nodes back to code and data.

10. Why it worked

Production choice: plan reading is a diagnostic skill. Record before/after plans when changing joins, partitioning or filters so improvements remain explainable.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you see a slow job but no plan evidence, capture `explain/UI` signals before changing configs.

C3-05-L11 - Now prove it yourself

13. Common beginner mistakes

- Reading only the top line of explain output.
- Tuning a slow job before capturing plan evidence.
- Assuming a physical plan proves the business result is correct.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Annotate a second plan where the dimension is small enough to consider broadcast.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: plan reading is a diagnostic skill. Record before/after plans when changing joins, partitioning or filters so improvements remain explainable.

18. Teach it back

Explain query plans, explain and execution-stage reasoning in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P11 evidence first. Then open C3-062 R11. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

annotated plan + changed plan. You can map a fresh plan node to its code/data cause.

C3-05-L12 - Caching and persistence decisions

Focused effort: 2.5 h | Matching practice: P12 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Cache only reusable expensive intermediates and unpersist deliberately; reject cache-everything habits.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture storing a prepared ingredient in a refrigerator. Caching helps if several dishes reuse that expensive preparation. If you only cook once, the storage and materialization cost may exceed the recomputation you hoped to save.

5. New words before we use them

cache = persist DataFrame with default storage level. persist = choose a storage level. materialize = force evaluation so cached blocks actually exist. unpersist = release cached data.

6. Useful source / visual

[Spark SQL performance tuning - caching](#)

[OPTIONAL VISUAL: Spark UI performance walkthrough](#)

Useful segment: Inspect storage/execution only after the book's cache decision logic.

C3-05-L12 - Follow with me once

7. Tiny worked example / prediction

Compare: raw source used once; clean joined fact reused by three reports; tiny dimension. Decide cache/do-not-cache and why.

8. Follow with me once - real project action

Complete docs/L12_cache_decisions.md and optional runtime cache demo.

9. Expected result

A cache decision matrix plus, when runtime is available, proof that a reused intermediate is materialized and later unpersisted.

10. Why it worked

Production choice: caching is an economic decision about recomputation versus memory/storage pressure. Document reuse count, materialization and cleanup.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If cache seems to make no difference, verify materialization and reuse count; tiny local jobs can hide realistic cost.

C3-05-L12 - Now prove it yourself

13. Common beginner mistakes

- Caching every DataFrame.
- Caching but never materializing/reusing it.
- Forgetting to unpersist long-lived intermediates.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Re-evaluate when the reused DataFrame is 20x larger than executor memory.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: caching is an economic decision about recomputation versus memory/storage pressure. Document reuse count, materialization and cleanup.

18. Teach it back

Explain caching and persistence decisions in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P12 evidence first. Then open C3-062 R12. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

decision note + changed-scale transfer. You can make a fresh cache decision from reuse/cost/memory evidence.

C3-05-L13 - Data skew and uneven partition diagnosis

Focused effort: 3.0 h | Matching practice: P13 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Measure hot-key and partition imbalance before selecting skew remedies.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture four checkout lanes where one lane receives 70% of customers. The other lanes finish quickly while one straggles. Skew diagnosis starts by proving key and partition imbalance and deciding whether the hot key is valid business concentration or dirty data.

5. New words before we use them

Skew = uneven key/partition/task workload. Straggler = task that runs much longer than peers. Salting = splitting a hot key with an added bucket; use cautiously. Hot key = disproportionately frequent key.

6. Useful source / visual

[Spark SQL performance tuning - skew](#)

[RECOMMENDED VISUAL: Spark UI performance walkthrough](#)

Useful segment: Use task-duration/partition evidence to recognize stragglers.

C3-05-L13 - Follow with me once

7. Tiny worked example / prediction

Profile region frequency where EAST intentionally dominates. Distinguish business-real hot key from bad-key concentration.

8. Follow with me once - real project action

Complete scripts/05_skew_profile_TODO.py and record max/median key frequency plus proposed least-invasive remedy.

9. Expected result

A key-frequency/partition diagnosis that identifies the hot key and distinguishes valid business skew from dirty-key concentration.

10. Why it worked

Production choice: diagnose skew at key, partition and task levels. Fix bad keys first; choose broadcast/pre-aggregation/AQE/salting only when their assumptions are met.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If you immediately salt, first prove the key is valid and that a simpler strategy cannot solve it.

C3-05-L13 - Now prove it yourself

13. Common beginner mistakes

- Salting immediately when one key is large.
- Ignoring that UNKNOWN/NULL concentration may be a quality defect.
- Using row counts only and not task/partition evidence.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Analyze a changed fixture where UNKNOWN contains 40% of rows and decide whether data quality is the real problem.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: diagnose skew at key, partition and task levels. Fix bad keys first; choose broadcast/pre-aggregation/AQE/salting only when their assumptions are met.

18. Teach it back

Explain data skew and uneven partition diagnosis in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P13 evidence first. Then open C3-062 R13. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

distribution evidence + remedy decision. You can diagnose fresh skew and choose a proportionate response.

C3-05-L14 - Adaptive execution and tuning concepts

Focused effort: 3.0 h | Matching practice: P14 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Explain what AQE can adapt using runtime statistics and tune from evidence rather than random configuration changes.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a route planner that can react after traffic conditions become visible. AQE can change supported physical decisions using runtime statistics, but it cannot change a wrong destination. Correct grain and logic still come first.

5. New words before we use them

AQE = Adaptive Query Execution. Runtime statistics = observed sizes/distributions during execution. Coalesce post-shuffle partitions = AQE can merge small partitions. Skew join optimization = split/handle skewed shuffle partitions when conditions are met.

6. Useful source / visual

[Spark SQL performance tuning - AQE](#)

[OPTIONAL VISUAL: Query plan to Spark SQL UI performance](#)

Useful segment: Compare planned versus observed/adaptive execution; no tuning by folklore.

C3-05-L14 - Follow with me once

7. Tiny worked example / prediction

Compare the configured AQE flag and capture adaptive plan markers on a shuffle workload.

8. Follow with me once - real project action

Complete docs/L14_aqe_experiment.md. State one hypothesis, one changed config and the evidence required to keep/revert it.

9. Expected result

A one-variable AQE/tuning experiment with correctness parity and before/after plan/runtime evidence.

10. Why it worked

Production choice: AQE is a helper, not a substitute for modeling. Run controlled experiments with one hypothesis and protect output correctness during tuning.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If a "tuning" change has no hypothesis or correctness check, revert and design a controlled comparison.

C3-05-L14 - Now prove it yourself

13. Common beginner mistakes

- Treating AQE as automatic correctness.
- Changing many configs at once.
- Keeping a tuning change without before/after evidence.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Consider a cluster-sized dataset where initial shuffle partitions are high and AQE coalesces at runtime.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: AQE is a helper, not a substitute for modeling. Run controlled experiments with one hypothesis and protect output correctness during tuning.

18. Teach it back

Explain adaptive execution and tuning concepts in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P14 evidence first. Then open C3-062 R14. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

controlled experiment note. You can defend one fresh AQE/tuning decision with before/after evidence.

C3-05-L15 - Testing Spark transformations and output invariants

Focused effort: 3.0 h | Matching practice: P15 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Test pure DataFrame transformations with deterministic fixtures plus invariants for grain, nulls, counts and totals.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture a safety harness around a transformation. A test can prove exact rows for a tiny fixture, while invariants protect broader rules such as unique keys, row reconciliation, schema shape and totals. Both matter because Spark code can be syntactically valid and logically wrong.

5. New words before we use them

`assertDataFrameEqual` = PySpark utility for comparing DataFrames. `assertSchemaEqual` = schema comparison utility. Fixture = small deterministic test input. Invariant = rule that must always hold even when exact rows change.

6. Useful source / visual

[Testing PySpark](#)

Book + official documentation are sufficient; no external video is required for mastery.

C3-05-L15 - Follow with me once

7. Tiny worked example / prediction

Test `clean_orders` on a tiny `DataFrame` and add invariants for unique `order_id`, rejected reconciliation and regional revenue total.

8. Follow with me once - real project action

Complete `tests/test_transforms_TODO.py` once PySpark is installed. Keep source-control tests runnable even without Spark.

9. Expected result

Passing deterministic transformation tests plus grain/schema/reconciliation invariants.

10. Why it worked

Production choice: keep transformation functions testable outside orchestration. Tests should cover exact examples plus invariants and failure behavior.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If a test only checks row count, add value/grain/schema or failure-behavior evidence.

C3-05-L15 - Now prove it yourself

13. Common beginner mistakes

- Testing only row count.
- Making tests depend on unordered row output.
- Skipping schema and rejection invariants.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Add a duplicate product fixture and assert the join validation blocks it.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: keep transformation functions testable outside orchestration. Tests should cover exact examples plus invariants and failure behavior.

18. Teach it back

Explain testing spark transformations and output invariants in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P15 evidence first. Then open C3-062 R15. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

pytest evidence + invariant matrix. You can write fresh deterministic Spark tests and explain what each failure protects.

C3-05-L16 - Packaging and running repeatable Spark jobs

Focused effort: 3.5 h | Matching practice: P16 | Prerequisite: prior lesson mastered/remediated; C3-04 Gate PASS is assumed.

1. Why it matters

Distributed correctness must remain explainable and reproducible; performance claims require plan/data/runtime evidence, not folklore.

2. What you learn

Package session/config, schemas, transforms, I/O and CLI boundaries so the same job can run from a clean environment.

3. Prerequisite / what you already know

You already know Python/SQL, explicit grain, ingestion, warehousing and lakehouse basics. This lesson teaches the new distributed-processing behavior; no prior Spark knowledge is assumed.

4. Picture it in your head

Picture handing a machine to another engineer. A notebook that only runs on your laptop is not a handoff. A repeatable Spark job separates session/config, schemas, transforms, I/O and CLI parameters so another machine can reconstruct the run.

5. New words before we use them

Entrypoint = command/module used to start the job. CLI = command-line interface. Parameter = input/output/date/config supplied at run time. Reproducibility = another clean environment can recreate the intended run and evidence.

6. Useful source / visual

[Spark submitting applications](#)

[OPTIONAL VISUAL: Data Engineering Zoomcamp Spark module](#)

Useful segment: Use packaging/submission examples only as reinforcement; learner package stays authoritative.

C3-05-L16 - Follow with me once

7. Tiny worked example / prediction

Run the packaged orders job with input/output arguments and save version, command, output tree and reconciliation.

8. Follow with me once - real project action

Finish scripts/02_orders_job_TODO.py, README, RUNBOOK and evidence template. Do not hardcode your own absolute path.

9. Expected result

A CLI-style Spark job that another clean environment can run from documented arguments and dependencies.

10. Why it worked

Production choice: package runtime assumptions, arguments, dependencies, logs, validations and runbook together. A teammate should not need your shell history to reproduce the job.

11. Break it on purpose

Change one assumption related to this lesson (schema/key/null/partition/config/reuse/input scale). Predict the failure or plan change first, then run/inspect. Keep the broken evidence beside the repaired evidence instead of deleting it.

12. Repair from evidence

If a teammate cannot rerun it from README, the phase is not finished even if your notebook works.

C3-05-L16 - Now prove it yourself

13. Common beginner mistakes

- Hardcoding local absolute paths.
- Shipping a notebook without CLI/runbook/dependencies.
- Calling a job repeatable without a clean-environment rerun.

14. Stuck? recovery ladder

1) Restate input/output grain. 2) Check schema and source controls. 3) Isolate the smallest transformation. 4) Compare expected and observed rows/keys/values. 5) Inspect formatted plan only after logic is right. 6) Change one physical variable at a time. 7) Open the protected review only after saving a genuine attempt.

15. Small independent changed task

Run or dry-run the same package against a fresh shipment dataset with different paths.

16. Validate

Save correctness controls first (row/key/value/rejected reconciliation). Then add partition/plan/runtime evidence appropriate to the lesson. A fast-looking wrong result is a fail.

17. Production reasoning

Production choice: package runtime assumptions, arguments, dependencies, logs, validations and runbook together. A teammate should not need your shell history to reproduce the job.

18. Teach it back

Explain packaging and running repeatable spark jobs in 60-90 seconds: what happens logically, what can happen physically, and what evidence would prove your claim.

19. Protected review + changed retry

Save P16 evidence first. Then open C3-062 R16. Close the review and solve a changed case with at least one different key/input/scale/config. Never mark mastery from reading the review.

20. Evidence / mastery

job package + handoff + changed-domain run. You can hand off a fresh Spark job with exact command, dependencies, tests and validation.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.